

1. Introducción

Hasta ahora, nuestras aplicaciones dependían de que Google, TMDB o Firebase nos dieran permiso y datos. Ahora, nosotros construiremos nuestra propia API

Una **API Propia** es un activo digital que encapsula las **Reglas de Negocio** de una empresa. No es solo "leer una base de datos"; es validar, procesar, asegurar y distribuir información a múltiples clientes (Web, Móvil, Relojes inteligentes, etc.).

2. Marco Teórico: Frameworks de Desarrollo

Seleccionar frameworks para agilizar el desarrollo.

¿Qué es un Framework Web?

Si desarrollar una aplicación fuera construir una casa:

- **JavaScript Puro (Node.js http):** Es fabricar tus propios ladrillos y cemento. Tienes control total, pero tardas años.
- **Framework (Express.js):** Te dan la estructura, las vigas, las paredes prefabricadas y la instalación eléctrica lista. Tú te encargas de la distribución y decoración.

Selección Tecnológica: Express.js

Para este curso utilizaremos **Express.js**.

- **Justificación:** Es el estándar de facto en la industria para Node.js. Es "no opinado" (un-opinionated), lo que significa que nos da libertad para definir nuestra propia arquitectura, ideal para aprender patrones de diseño.
-

3. Conceptos Críticos de una API Robusta

Para desarrollar una interfaz profesional, no basta con que funcione; debe ser **segura** y **escalable**.

A. Seguridad con Tokens (JWT - JSON Web Tokens)

Las APIs REST son **Stateless** (sin estado). El servidor no "recuerda" al usuario entre una petición y otra (a diferencia de las sesiones con cookies antiguas).

El Problema: ¿Cómo sabe el servidor que quien pide "borrar producto" es el administrador que se logueó hace 5 minutos?

La Solución (JWT): Cuando el usuario hace login, el servidor le entrega una "Credencial Digital Firmada" (el Token).

1. **Header:** Dice qué algoritmo de encriptación se usa.
2. **Payload:** Contiene los datos (ID de usuario, Rol: Admin).
3. **Signature:** Una firma matemática creada con una **Clave Secreta** que solo el servidor conoce.

En cada petición subsiguiente, el cliente (React) envía este token. Si un hacker intenta modificar el token (ej. cambiarse el rol a Admin), la firma matemática se romperá y el servidor rechazará la petición.

B. CORS (Cross-Origin Resource Sharing)

Este es el error #1 que frustra a los desarrolladores Frontend.

El Concepto: Por seguridad, los navegadores bloquean por defecto que una web ([mi-tienda.com](#)) pida datos a un servidor diferente ([api-servidor.com](#)). Esto es para evitar que una página maliciosa lea tus datos bancarios.

La Implementación: En nuestra API, debemos configurar explícitamente **CORS** para decirle al navegador: *"yo autorizo que la página [localhost:5173](#) (donde estará React) consuma mis datos"*.

Diseño de APIs RESTful

1. El Concepto de "Recurso"

En una arquitectura REST (Representational State Transfer), no pensamos en "páginas" ni en "funciones", pensamos en **Recursos**.

Un recurso es cualquier entidad de información que se puede nombrar:

- Un usuario.
- Un producto.
- Una orden de compra.

2. Los Verbos HTTP (Semántica)

El error de novato más común es usar **POST** para todo. El protocolo HTTP tiene "verbos" que indican la **intención** de la acción. Usarlos correctamente hace que tu API sea *semántica* y predecible.

Verbo	Acción (CRUD)	Descripción Técnica	
GET	Read (Leer)	Solicita una representación de un recurso. Nunca debe modificar datos.	
POST	Create (Crear)	Envía datos para crear un nuevo recurso subordinado.	
PUT	Update (Reemplazar)	Actualiza un recurso reemplazándolo completamente . Si envías solo el nombre, se borra el resto.	
PATCH	Update (Modificar)	Aplica modificaciones parciales . Si envías solo el nombre, solo cambia el nombre y conserva lo demás.	
DELETE	Delete (Borrar)	Elimina el recurso especificado.	

3. Buenas Prácticas en Naming (Rutas y Endpoints)

La estructura de tus URLs dice mucho de tu calidad como desarrollador.

Regla A: Usa Sustantivos, NO Verbos

La URL identifica el **recurso**, el verbo HTTP identifica la **acción**.

-  **Mal:** `/api/crearProducto`, `/api/obtenerUsuarios`, `/api/borrar-orden`
-  **Bien:** `/api/productos`, `/api/usuarios`, `/api/ordenes`

Regla B: Usa Plurales

Por convención, la ruta base siempre es plural porque representa la colección entera.

- ❌ **Mal:** `/api/producto/501`
- ✅ **Bien:** `/api/productos/501` (Léelo como: "De la lista de productos, dame el 501")

Regla C: Jerarquía Lógica

Si un recurso depende de otro, la URL debe reflejarlo.

- **Ejemplo:** Quiero ver las órdenes del usuario 42.
- ✅ **Ruta:** `GET /api/usuarios/42/ordenes/3`

Regla D: Filtrado, Paginación y Ordenamiento

No crees rutas nuevas para filtros. Usa **Query Parameters** (parámetros de consulta).

- ❌ **Mal:** `/api/productos-baratos`
 - ✅ **Bien:** `GET /api/productos?precio_max=500&orden=desc`
-

4. Los Códigos de Estado HTTP (El Semáforo)

Tu API no solo debe devolver datos JSON, debe devolver un **Status Code** correcto para que el navegador sepa qué pasó.

2xx: Éxito

- **200 OK:** Todo salió bien (Estándar para GET, PUT, PATCH).
- **201 Created:** Éxito y se creó algo nuevo (Estándar obligatorio para **POST**).
- **204 No Content:** Éxito, pero no hay nada que devolver (Común en **DELETE**).

3xx: Redirección

- **304 Not Modified:** Usado para caché. "El dato no ha cambiado desde la última vez que preguntaste".

4xx: Error del Cliente (Culpa tuya/del Frontend)

- **400 Bad Request:** Enviaste datos mal formados o faltantes (ej. faltó el precio).
- **401 Unauthorized:** No tienes llave de entrada (Token JWT falso o inexistente).
- **403 Forbidden:** Tienes llave, pero no tienes permiso para entrar aquí (ej. Un usuario normal queriendo borrar productos).
- **404 Not Found:** El recurso (ID) no existe.

5xx: Error del Servidor (Culpa nuestra/del Backend)

- **500 Internal Server Error:** Explotó el código (bug en el servidor, base de datos caída).
-

5. Estructura de Respuesta (El Contrato JSON)

Define una estructura estándar y úsala siempre.

Ejemplo de Estructura Recomendada para EcoMarket:

```
// Opción A: Respuesta Exitosa
{
  "success": true,
  "data": [ ... array de productos ... ],
  "count": 15, // Útil para paginación
  "message": "Productos cargados correctamente"
}

// Opción B: Respuesta de Error
{
  "success": false,
  "error_code": "PRODUCT_OUT_OF_STOCK", // Código interno para que React sepa qué
  mostrar
  "message": "No hay suficiente stock para realizar la venta" // Mensaje humano
}
```

6. Seguridad: CORS y Headers

- **CORS (Cross-Origin Resource Sharing):** Es un mecanismo de seguridad del navegador. Por defecto, una web en [dominio-a.com](#) no puede pedir datos a [dominio-b.com](#).
 - *En desarrollo:* Nuestro React estará en [localhost:5173](#) y nuestro Node en [localhost:3000](#). Son "orígenes distintos". Debemos configurar el servidor para que acepte peticiones de ese puerto específico.
 - **Headers de Seguridad (Helmet):** En producción, una API debe enviar cabeceras HTTP especiales para evitar ataques comunes como XSS (Cross-Site Scripting) o Sniffing. Usaremos la librería [helmet](#) para esto.
-

EcoMarket API.

Fase 1: Arquitectura Relacional y Configuración del Entorno

1. Introducción al Stack Tecnológico

Para el desarrollo de la API **EcoMarket**, utilizaremos una arquitectura robusta basada en el modelo relacional.

Las Herramientas:

- **Node.js & Express:** El entorno de ejecución y el framework para manejar las peticiones HTTP.
- **MySQL / MariaDB:** El motor de base de datos donde se guardará la información en tablas.
- **Sequelize (ORM):** Una librería que nos permite interactuar con la base de datos usando objetos de JavaScript (Clases y Métodos) en lugar de escribir consultas SQL manuales.

2. Preparación de la Base de Datos

Antes de escribir código, debemos preparar el contenedor de datos.

Instrucción:

1. Abre tu cliente de base de datos favorito (phpMyAdmin, DBeaver, MySQL Workbench o HeidiSQL).
2. Ejecuta la siguiente sentencia SQL para crear la base de datos vacía:

```
CREATE DATABASE ecomarket_db;
```

3. Inicialización del Proyecto

1. Crea una carpeta nueva llamada **EcoMarket-API**.

Abre la terminal en esa carpeta y ejecuta:

```
npm init -y
```

2. Instalación de Dependencias de Producción:

```
npm install express sequelize mysql2 dotenv cors helmet
```

- **mysql2**: Es el "driver" (conductor) de bajo nivel que permite a Node.js conectarse al puerto de MySQL.
- **sequelize**: Es el ORM que administra los modelos y relaciones.
-

4 .Instalación de Dependencias de Desarrollo:

```
npm install -D nodemon
```

4. Estructura Profesional de Carpetas (MVC)

Implementaremos el patrón **Modelo-Vista-Controlador** (adaptado a API) para mantener el código organizado y escalable. Crea manualmente la siguiente estructura:

```
/EcoMarket-API
├── .env           <-- Variables de entorno (Credenciales)
├── .gitignore     <-- Archivo de exclusión para Git
├── server.js      <-- Punto de entrada de la aplicación
└── /src           <-- Código Fuente
    ├── /config     <-- Configuración de la Base de Datos
    ├── /controllers <-- Lógica de negocio (Funciones)
    ├── /models     <-- Definición de Tablas (Modelos Sequelize)
    ├── /routes     <-- Definición de Endpoints (URLs)
    └── /middlewares <-- Seguridad y validaciones
```

5. Configuración de Seguridad (.env)

En la raíz del proyecto, crea un archivo llamado `.env`. Aquí definiremos las credenciales de acceso para que no estén colocadas en el código fuente.

```
PORT=4000
DB_NAME=ecomarket_db
DB_USER=root
DB_PASS= # Pon aquí tu contraseña de MySQL (en XAMPP suele ser
vacío)
DB_HOST=localhost
DB_DIALECT=mysql # Cambia a 'mariadb' si usas ese motor
específicamente
```

Importante: Crea también un archivo `.gitignore` y agrega:

```
node_modules
.env
```

6. Módulo de Conexión (`src/config/database.js`)

Este archivo será el responsable de crear la instancia de conexión usando **Sequelize**.

Archivo: `src/config/database.js`

```
const { Sequelize } = require('sequelize');

// Cargar variables de entorno
require('dotenv').config();

// Crear la instancia de conexión
const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASS,
  {
    host: process.env.DB_HOST,
    dialect: process.env.DB_DIALECT, // 'mysql' o 'mariadb'
    logging: false // Cambiar a true para ver las consultas SQL en la consola
  }
);

// Función asíncrona para verificar la conexión
const testConnection = async () => {
  try {
    await sequelize.authenticate();
    console.log('Conexión establecida con la Base de Datos.');
```

7. El Servidor Principal (**server.js**)

Este es el archivo principal que ejecutará Node.js. Su función es iniciar Express, configurar la seguridad y sincronizar la base de datos.

Archivo: **server.js** (En la raíz)

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const { sequelize, testConnection } = require('./src/config/database');

const app = express();
const PORT = process.env.PORT || 4000;

// 1. Middlewares (Configuración Global)
app.use(helmet()); // Añade cabeceras de seguridad HTTP
app.use(cors()); // Permite peticiones desde otros dominios
app.use(express.json()); // Permite recibir datos JSON en el cuerpo de las peticiones

// 2. Ruta de Prueba (Health Check)
app.get('/', (req, res) => {
  res.json({
    status: "API Online",
    engine: "MySQL/Sequelize",
    version: "1.0.0"
  });
});

// 3. Inicialización del Sistema
const startServer = async () => {
  try {
    // A) Verificar conexión con la BD
    await testConnection();

    // B) Sincronización de Modelos
    // Esta función revisa los modelos y crea las tablas si no existen.
    // { force: false } asegura que NO borre los datos existentes al reiniciar.
    await sequelize.sync({ force: false });
    console.log(' Tablas sincronizadas correctamente. ');

    // C) Levantar el servidor Express
    app.listen(PORT, () => {
      console.log(` Servidor corriendo en http://localhost:${PORT} `);
    });
  } catch (error) {
    console.error(' Error fatal al iniciar el servicio:', error);
  }
};

startServer();

```

8. Ejecución y Validación

Abre el archivo `package.json` y modifica la sección `scripts` para facilitar la ejecución:

```
"scripts": {  
  "dev": "nodemon server.js",  
  "start": "node server.js"  
},
```

En la terminal, ejecuta:

```
npm run dev
```

Nota: Ejecuta el script llamado dev que esté definido en el archivo:

1. Resultado esperado en consola:

```
[nodemon] starting `node server.js`  
Conexión establecida con la Base de Datos.  
Executing (default): SELECT 1+1 AS result  
Tablas sincronizadas correctamente.  
Servidor corriendo en http://localhost:4000
```

Actividad “Auditoría de Conexión”

Para asegurar que la configuración es correcta realiza la siguiente actividad:

1. **Cambio de Puerto:** Detén el servidor. Modifica el archivo `.env` para usar el puerto `5050` en lugar del 4000.
2. **Prueba de Fallo:** Modifica el `.env` poniendo un usuario de base de datos incorrecto (ej. `DB_USER=usuario_falso`).
 - Ejecuta el servidor y observa el error `Access denied` en la consola. Esto confirma que `sequelize` está intentando conectar realmente.
 - Corrige el usuario para que vuelva a funcionar.
3. **Endpoint de Estado:** Agrega una nueva ruta `GET /api/status` en `server.js` que devuelva un JSON con tu nombre completo y la fecha actual del servidor.
 - **Entregable:** Captura de pantalla del navegador accediendo a `http://localhost:5050/api/status`.

Fase 2: Modelado de Usuarios y Autenticación Segura

1. Marco Teórico: Seguridad Digital

A. Nunca guardes contraseñas en texto plano

Si guardas la contraseña "123456" tal cual en la base de datos, y alguien hackea tu tabla, tendrá acceso a las cuentas de tus usuarios (que probablemente usan esa misma contraseña en su correo). **Solución:** Usaremos **Hashing** (con la librería `bcryptjs`). Convertiremos "123456" en algo como `$2b$10$EixZaYVK1fsdf...`. Es un proceso unidireccional: puedes convertir texto a hash, pero no hash a texto.

B. El Carnet Digital (JWT)

Como nuestra API es REST (sin estado), el servidor no recuerda si ya iniciaste sesión.

Solución: Cuando el usuario hace Login correctamente, le entregamos un **Token JWT** (JSON Web Token).

- Es como un brazalete de hotel "Todo Incluido".
- El usuario debe enviar este token en el **Header** de cada petición (`Authorization: Bearer <token>`).
- Si el token es válido, el servidor le deja pasar (CORS y Auth Middleware).

2. Instalación de Dependencias de Seguridad

Necesitamos dos herramientas nuevas. Abre la terminal en `EcoMarket-API` y ejecuta:

```
npm install bcryptjs jsonwebtoken
```

- `bcryptjs`: Para encriptar contraseñas.
- `jsonwebtoken`: Para generar y verificar tokens.

Además, agrega una clave secreta en tu archivo `.env` (invéntate una frase larga):

```
# ... (variables anteriores) ...  
JWT_SECRET=MiFraseSecretaSuperSeguraParaEcoMarket2024
```

JWT_EXPIRES=1d

3. El Modelo de Usuario (`src/models/User.js`)

Aquí definiremos la tabla de usuarios usando Sequelize. Aprenderemos algo avanzado: **Hooks**. Un Hook es una función que se dispara automáticamente *antes* de guardar en la BD. Lo usaremos para encriptar la contraseña automáticamente.

Archivo: `src/models/User.js`

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');
const bcrypt = require('bcryptjs');

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true, // No permitir correos repetidos
    validate: {
      isEmail: true // Validación automática de Sequelize
    }
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false
  },
  rol: {
    type: DataTypes.ENUM('user', 'admin'),
    defaultValue: 'user'
  }
}, {
  hooks: {
    beforeCreate: async (user) => {
      user.password = await bcrypt.hash(user.password, 10);
    }
  }
});
```

```
hooks: {  
  // HOOK: Antes de crear el usuario, encriptamos la clave  
  beforeCreate: async (user) => {  
    const salt = await bcrypt.genSalt(10);  
    user.password = await bcrypt.hash(user.password, salt);  
  }  
}  
});  
  
// Método personalizado para comparar contraseñas (Login)  
User.prototype.compararPassword = async function(passwordIngresada) {  
  return await bcrypt.compare(passwordIngresada, this.password);  
};  
  
module.exports = User;
```

4. El Controlador de Autenticación (src/controllers/auth.controller.js)

Aquí va la lógica de negocio: Registro y Login.

Archivo: src/controllers/auth.controller.js

```
const User = require('../models/User');
const jwt = require('jsonwebtoken');

// Función auxiliar para crear el token
const generarToken = (id, rol) => {
  return jwt.sign({ id, rol }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES
  });
};

exports.registro = async (req, res) => {
  try {
    const { nombre, email, password } = req.body;

    // 1. Verificar si ya existe
    const existeUsuario = await User.findOne({ where: { email } });
    if (existeUsuario) {
      return res.status(400).json({ error: 'El correo ya está registrado' });
    }

    // 2. Crear usuario (El hook beforeCreate encriptará el password)
    const nuevoUsuario = await User.create({ nombre, email, password });

    // 3. Generar token
    const token = generarToken(nuevoUsuario.id, nuevoUsuario.rol);

    res.status(201).json({
      success: true,
      token,
      usuario: { id: nuevoUsuario.id, nombre: nuevoUsuario.nombre, email: nuevoUsuario.email }
    });
  }
};
```



```

    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  };

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;

    // 1. Buscar usuario
    const usuario = await User.findOne({ where: { email } });
    if (!usuario) {
      return res.status(401).json({ error: 'Credenciales inválidas' });
    }

    // 2. Comparar contraseña (usando el método que creamos en el modelo)
    const esCorrecto = await usuario.compararPassword(password);
    if (!esCorrecto) {
      return res.status(401).json({ error: 'Credenciales inválidas' });
    }

    // 3. Generar token
    const token = generarToken(usuario.id, usuario.rol);

    res.json({
      success: true,
      token,
      usuario: { id: usuario.id, nombre: usuario.nombre, role: usuario.rol }
    });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

```

5. Las Rutas (src/routes/auth.routes.js)

Definimos las URLs.

Archivo: src/routes/auth.routes.js

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/auth.controller');

// Definir los endpoints
// POST http://localhost:4000/api/auth/registro
router.post('/registro', authController.registro);

// POST http://localhost:4000/api/auth/login
router.post('/login', authController.login);

module.exports = router;
```

6. Middleware de Protección

(src/middlewares/auth.middleware.js)

Este es el "Guardia de Seguridad". Lo usaremos en la Fase 3.

Verifica que el token venga en la cabecera.

Archivo: src/middlewares/auth.middleware.js

```

const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.protect = async (req, res, next) => {
  let token;

  // 1. Verificar si existe el header "Authorization: Bearer <token>"
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1]; // Sacamos el token

      // 2. Verificar la firma del token
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      // 3. Buscar al usuario dueño del token y adjuntarlo a la request
      req.user = await User.findById(decoded.id);

      next(); // Dejar pasar
    } catch (error) {
      res.status(401).json({ error: 'Token no válido o expirado' });
    }
  }

  if (!token) {
    res.status(401).json({ error: 'Acceso denegado, no enviaste token' });
  }
};

```

7. Integración Final en `server.js`

Finalmente, avisamos al servidor que existen estas nuevas rutas y modelos.

Abre `server.js` y modifica/añade lo siguiente:

```

// ... imports anteriores ...
const authRoutes = require('./src/routes/auth.routes'); // <-- IMPORTAR RUTAS
require('./src/models/User'); // <-- IMPORTAR EL MODELO PARA QUE SEQUELIZE LO CREE

const app = express();
// ... middlewares ...

// RUTAS
app.use('/api/auth', authRoutes); // <-- MONTAR RUTAS

// ... startServer ...

```

8. Ejecución y Prueba

1. Reinicia el servidor (`npm run dev`).
 - *Deberías ver en consola:* `Executing (default): CREATE TABLE IF NOT EXISTS Users ...` (Sequelize creando la tabla).
2. Abre **Postman** (o Thunder Client en VS Code).

Prueba A: Registro

- Método: `POST`
- URL: `http://localhost:4000/api/auth/registro`

Body (JSON):

```
{
  "nombre": "Estudiante Demo",
  "email": "alumno@test.com",
  "password": "password123"
}
```

- *Resultado:* Debería devolver un `token`.

Prueba B: Verificación en Base de Datos

- Ve a tu gestor SQL.
- Haz un `SELECT * FROM Users;`
- Mira la columna `password`. ¡No dice "password123"!
- Debe de decir algo como (`$2a$10...`).

Actividad para el Alumno: "El Endpoint del Perfil"

Ahora que tienen el middleware de seguridad (`protect`), deben usarlo.

Instrucción:

1. En `auth.controller.js`, crea una función `miPerfil` que devuelva `res.json(req.user)`.
2. En `auth.routes.js`, crea la ruta `GET /perfil`.

El Reto: Protege esa ruta agregando el middleware `protect` antes del controlador.

```
// Pista
const { protect } = require('../middlewares/auth.middleware');
router.get('/perfil', protect, authController.miPerfil);
|
```

3. **Prueba:** Intentar entrar a `/perfil` sin token (debe fallar) y con token (debe mostrar sus datos).

Fase 3. Lógica de Negocio (Productos, Categorías e Imágenes)

1. Instalación de Dependencias (Gestión de Archivos)

Para manejar la subida de imágenes (multipart/form-data), utilizaremos el estándar de la industria en Node.js: **Multer**.

Abre la terminal en `EcoMarket-API` y ejecuta:

```
npm install multer
```

Nota: Crea manualmente una carpeta llamada `uploads` en la raíz de tu proyecto. Aquí es donde Multer guardará las fotos.

2. Modelado de Datos Relacional

En SQL, el orden importa. No podemos crear la tabla `Products` si no existe la tabla `Categories`, porque el producto necesita un `category_id`.

Paso 2.1: El Modelo de Categoría (`src/models/Category.js`)

Archivo: `src/models/Category.js`

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const Category = sequelize.define('Category', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  }
}, {
  timestamps: false // No necesitamos createdAt/updatedAt para categorías simples
});

module.exports = Category;
```

Paso 2.2: El Modelo de Producto (`src/models/Product.js`)

Aquí definiremos la **Clave Foránea (Foreign Key)**.

Archivo: `src/models/Product.js`

```

const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');
const Category = require('../Category'); // Importamos para relacionar

const Product = sequelize.define('Product', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  },
  precio: {
    type: DataTypes.DECIMAL(10, 2), // 10 dígitos, 2 decimales
    allowNull: false
  },
  stock: {
    type: DataTypes.INTEGER,
    defaultValue: 0
  },
  descripcion: {
    type: DataTypes.TEXT
  },
  imagen_url: {
    type: DataTypes.STRING, // Guardaremos la RUTA del archivo, no el archivo binario
    allowNull: true
  }
});

// DEFINICIÓN DE RELACIONES (ASSOCIATIONS)
// Una Categoría tiene muchos Productos
Category.hasMany(Product);
// Un Producto pertenece a una Categoría
Product.belongsTo(Category);

module.exports = Product;

```

3. Configuración de Subida de Archivos (src/middlewares/upload.middleware.js)

Crearemos un middleware reutilizable que se encargue de:

1. Recibir el archivo.
2. Renombrarlo (para evitar colisiones si dos usuarios suben `foto.jpg`).
3. Guardarlo en la carpeta `uploads`.

Archivo: `src/middlewares/upload.middleware.js`

```
const multer = require('multer');
const path = require('path');

// Configuración de almacenamiento
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/'); // Carpeta destino
  },
  filename: (req, file, cb) => {
    // Generamos un nombre único: producto-TIMESTAMP.extensión
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    const ext = path.extname(file.originalname);
    cb(null, file.fieldname + '-' + uniqueSuffix + ext);
  }
});

// Filtro para aceptar solo imágenes
const fileFilter = (req, file, cb) => {
  if (file.mimetype.startsWith('image/')) {
    cb(null, true);
  } else {
    cb(new Error('No es una imagen! Por favor sube solo imágenes.'), false);
  }
};

const upload = multer({ storage: storage, fileFilter: fileFilter });

module.exports = upload;
```

4. Controladores de Producto

`(src/controllers/product.controller.js)`

Aquí implementaremos el CRUD. Observa cómo manejamos la imagen (`req.file`).

Archivo: `src/controllers/product.controller.js`


```
const Product = require('../models/Product');
const Category = require('../models/Category');

// 1. CREAM PRODUCTO (POST)
exports.createProduct = async (req, res) => {
  try {
    const { nombre, precio, stock, description, CategoryId } = req.body;

    // Si se subió una imagen, obtenemos su ruta
    // Si no, asignamos null
    const imagen_url = req.file ? req.file.path : null;

    const newProduct = await Product.create({
      nombre,
      precio,
      stock,
      descripcion: description,
      imagen_url, // Sequelize guardará "uploads/producto-123.jpg"
      CategoryId // Clave foránea
    });

    res.status(201).json(newProduct);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

```
// 2. OBTENER TODOS LOS PRODUCTOS (GET)
exports.getProducts = async (req, res) => {
  try {
    const products = await Product.findAll({
      include: Category // JOIN automático con la tabla Categorías
    });
    res.json(products);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// 3. OBTENER UN PRODUCTO POR ID (GET)
exports.getProductById = async (req, res) => {
  try {
    const product = await Product.findByPk(req.params.id, {
      include: Category
    });
    if (!product) return res.status(404).json({ error: 'Producto no encontrado' });
    res.json(product);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

5. Rutas de Productos

(src/routes/product.routes.js)

Conectamos las rutas con los controladores y middlewares. **OJO:** Aquí usamos el `upload.single('imagen')`. Esto significa que Postman/Frontend debe enviar un campo llamado `imagen`.

Archivo: `src/routes/product.routes.js`

```
const express = require('express');
const router = express.Router();
const productController = require('../controllers/product.controller');
const upload = require('../middlewares/upload.middleware');
const { protect } = require('../middlewares/auth.middleware');

// Rutas Públicas
router.get('/', productController.getProducts);
router.get('/:id', productController.getProductById);

// Rutas Privadas (Solo usuarios logueados pueden crear)
// Nota el orden: 1. Proteger ruta -> 2. Procesar archivo -> 3. Controlador
router.post('/', protect, upload.single('imagen'), productController.createProduct);

module.exports = router;
```

6. Integración en el Servidor (**server.js**)

Tenemos que hacer dos cosas vitales aquí:

1. Registrar las rutas nuevas.
2. **Hacer pública la carpeta uploads**. Si no hacemos esto, el Frontend (React) no podrá ver las fotos (Error 403 Forbidden).

Modifica **server.js**:

```
// ... imports ...
const productRoutes = require('./src/routes/product.routes');
const path = require('path');

// ... middlewares ...

// 1. CARPETA PÚBLICA DE IMÁGENES
// Esto permite acceder a: http://localhost:4000/uploads/foto.jpg
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// 2. RUTAS
app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);

// ... sync & listen ...
```

7. Ejecución y Prueba (Postman)

Reinicia el servidor (`npm run dev`). Sequelize creará las tablas `Categories` y `Products` automáticamente.

Paso A: Crear una Categoría (Manual) Como no hicimos controlador de categorías, insértala directo en tu BD SQL:

```
INSERT INTO Categories (nombre) VALUES ('Tecnología');
```

Paso B: Login y Token Obtén tu token de la Fase 2 (`POST /api/auth/login`).

Paso C: Crear Producto con Imagen (Postman)

1. Método: **POST** `http://localhost:4000/api/products`
2. Headers: `Authorization: Bearer <TU_TOKEN>`
3. **Body**: Selecciona **form-data** (No raw JSON).
 - Key: `nombre` | Value: `Laptop Gamer`
 - Key: `precio` | Value: `15000`
 - Key: `CategoryId` | Value: `1`
 - Key: `imagen` | Value: [Selecciona Archivo] (Elige una foto jpg/png)
4. Send.

Paso D: Verificación

1. Revisa la carpeta `uploads` en tu proyecto. ¿Está la foto ahí?
 2. Copia la ruta que te devolvió el JSON (ej. `uploads/imagen-123.jpg`).
 3. Ve al navegador: `http://localhost:4000/uploads/imagen-123.jpg`.
¡Deberías ver la foto!
-

Actividad para el Alumno: "Completando el CRUD"

El controlador actual solo tiene `Create` y `Read`. **Reto:** Implementar la función `deleteProduct`.

1. En `product.controller.js`, crear `exports.deleteProduct`.
 - Debe buscar el producto por ID.
 - Debe verificar si existe.
 - Debe ejecutar `await product.destroy()`.
 - *(Bonus)*: Usar el módulo `fs` (File System) de Node.js para borrar también la imagen de la carpeta `uploads` y no dejar basura.
2. Agregar la ruta `DELETE /:id` en `product.routes.js` (protegida con token).
3. Entregar captura de Postman eliminando un producto exitosamente.

Fase 4. Órdenes de Compra y Transacciones (ACID)

1. Modelado de Datos Avanzado

Necesitamos dos tablas nuevas.

- **Order (Cabecera)**: Quién compró, cuándo, cuánto pagó en total y el estado (pendiente, pagado).
- **OrderItem (Detalle)**: Qué productos compró, cuántos de cada uno y a qué precio unitario.

Paso 1.1: Modelo de Orden (`src/models/Order.js`)

Archivo: `src/models/Order.js`

```

const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const Order = sequelize.define('Order', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  total: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false
  },
  estado: {
    type: DataTypes.ENUM('pendiente', 'completado', 'cancelado'),
    defaultValue: 'pendiente'
  }
}, {
  // El userId se agrega automáticamente con las relaciones
});

module.exports = Order;

```

Paso 1.2: Modelo de Ítem de Orden ([src/models/OrderItem.js](#))

Esta es la **Tabla Pivote** que genera la relación muchos-a-muchos entre Órdenes y Productos.

Archivo: [src/models/OrderItem.js](#)

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const OrderItem = sequelize.define('OrderItem', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  cantidad: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 1
  },
  precio_unitario: { // Importante: Guardamos el precio al momento de la compra
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false
  }
});

module.exports = OrderItem;
```

2. Configuración de Relaciones (src/models/associations.js)

Para no ensuciar `server.js` ni tener referencias circulares, crearemos un archivo dedicado a conectar todo el diagrama de la base de datos.

Archivo: `src/models/associations.js`

```
const User = require('./User');
const Product = require('./Product');
const Order = require('./Order');
const OrderItem = require('./OrderItem');

// 1. Usuario <-> Orden
// Un usuario tiene muchas órdenes
User.hasMany(Order);
// Una orden pertenece a un usuario
Order.belongsTo(User);

// 2. Orden <-> Producto (Muchos a Muchos)
// Una Orden tiene muchos Productos, A TRAVÉS de la tabla OrderItem
Order.belongsToMany(Product, { through: OrderItem });
// Un Producto puede estar en muchas Órdenes
Product.belongsToMany(Order, { through: OrderItem });

module.exports = { User, Product, Order, OrderItem };
|
```

3. El Controlador de Órdenes (Transacciones)

Aquí aplicaremos el concepto de **Transacción SQL**. Una transacción asegura que todas las operaciones se hagan juntas. Si una falla (ej. no hay stock de un producto), **todas se deshacen**, asegurando que no queden órdenes "a medias".

Archivo: `src/controllers/order.controller.js`


```
const { Order, Product, OrderItem } = require('../models/associations');
const { sequelize } = require('../config/database');

exports.createOrder = async (req, res) => {
  // Iniciamos una Transacción (ACID)
  const t = await sequelize.transaction();

  try {
    const { items } = req.body; // El cliente envía: [{ productId: 1, cantidad: 2 }, ...]
    const userId = req.user.id; // Viene del token

    let totalOrden = 0;
    const itemsParaCrear = [];

    // 1. Iterar sobre los productos solicitados
    for (const item of items) {
      const productoDB = await Product.findByPk(item.productId, { transaction: t });

      if (!productoDB) {
        throw new Error(`Producto con ID ${item.productId} no encontrado`);
      }

      // 2. Verificar Stock
      if (productoDB.stock < item.cantidad) {
        throw new Error(`Stock insuficiente para: ${productoDB.nombre}`);
      }

      // 3. Calcular subtotal y restar stock
      totalOrden += productoDB.precio * item.cantidad;
    }
  }
}
```

```

    // Restamos stock (dentro de la transacción)
    await productoDB.update({
      stock: productoDB.stock - item.cantidad
    }, { transaction: t });

    // Preparamos el objeto para OrderItem
    itemsParaCrear.push({
      cantidad: item.cantidad,
      precio_unitario: productoDB.precio, // Precio congelado
      ProductId: productoDB.id // FK
    });
  }

  // 4. Crear la Cabecera de la Orden
  const nuevaOrden = await Order.create({
    total: totalOrden,
    UserId: userId,
    estado: 'completado'
  }, { transaction: t });

  // 5. Crear los Detalles (Items) vinculados a la Orden
  for (const itemData of itemsParaCrear) {
    await OrderItem.create({
      ...itemData,
      OrderId: nuevaOrden.id
    }, { transaction: t });
  }

```

```

        // 6. Confirmar Transacción (COMMIT)
        await t.commit();

        res.status(201).json({
            success: true,
            message: "Compra realizada",
            orderId: nuevaOrden.id,
            total: totalOrden
        });
    } catch (error) {
        // 7. Si algo falló, deshacer todo (ROLLBACK)
        await t.rollback();
        res.status(400).json({
            success: false,
            error: error.message
        });
    }
};

// Historial de compras del usuario
exports.getMyOrders = async (req, res) => {
    try {
        const orders = await Order.findAll({
            where: { UserId: req.user.id },
            include: {
                model: Product,
                through: { attributes: ['cantidad', 'precio_unitario'] } // Traer datos de la tabla pivote
            }
        });
        res.json(orders);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
};

```

4. Rutas de Órdenes (src/routes/order.routes.js)

Archivo: src/routes/order.routes.js

```
const express = require('express');
const router = express.Router();
const orderController = require('../controllers/order.controller');
const { protect } = require('../middlewares/auth.middleware');

// Todas las rutas de órdenes requieren estar logueado
router.post('/', protect, orderController.createOrder);
router.get('/mis-compras', protect, orderController.getMyOrders);

module.exports = router;
```

5. Integración Final (server.js)

Tenemos que cargar las asociaciones **antes** de sincronizar la BD.

Modificar **server.js**:

```
// ... imports ...
const orderRoutes = require('./src/routes/order.routes');
require('./src/models/associations'); // <-- IMPORTANTE: Cargar relaciones

// ... middlewares ...

// Rutas
app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);
app.use('/api/orders', orderRoutes); // <-- Nueva ruta

// ... sync & listen ...
```

6. Ejecución y Prueba (Postman)

Esta es la prueba de fuego.

1. Reinicia el servidor (**npm run dev**). Sequelize creará **Orders** y **OrderItems**.
2. **Preparación:** Asegúrate de tener:
 - Un Usuario registrado y logueado (Token a la mano).
 - Un Producto creado (ID: 1) con Stock suficiente (ej. 100).

Prueba: Crear Orden

- **Método:** POST `http://localhost:4000/api/orders`
- **Header:** `Authorization: Bearer <TOKEN>`

Body (JSON):

JSON

```
{
  "items": [
    { "productId": 1, "cantidad": 2 },
  ]
}
```

Resultado Esperado:

1. JSON con `success: true` y el total calculado.
2. Si revisas la tabla `Products`, el stock debe haber bajado de 100 a 98.
3. Si intentas comprar 500 unidades (más de las que hay), debe dar error y **no** restar nada.

Actividad Integradora

Entregar el "Flow Completo de E-commerce":

1. **Registro:** Crear usuario "Cliente Final".
2. **Admin:** Entrar directo a la BD y cambiarle el rol a un usuario para que sea "Admin"
3. **Catálogo:** El Admin crea 3 productos con foto (middleware para creación/borrado de productos).
4. **Compra:** El Cliente compra 2 productos diferentes en una sola orden.
5. **Evidencia:** Captura del endpoint `GET /api/orders/mis-compras` mostrando el JSON anidado (Orden -> Productos).

Complemento:

Paginación y Filtrado de Productos

Modificaremos el controlador `getProducts` para que no devuelva todo de golpe.

Modificar: `src/controllers/product.controller.js`

```
const { Op } = require('sequelize'); // Operadores de Sequelize (para búsquedas)

exports.getProducts = async (req, res) => {
  try {
    // 1. Obtener parámetros de la URL (Query Params)
    // Ejemplo: ?page=1&limit=10&nombre=Laptop&minPrice=1000
    const { page = 1, limit = 10, nombre, minPrice, maxPrice } = req.query;

    // 2. Configurar Paginación
    const offset = (page - 1) * limit; // Cuántos saltar
    const limitNum = parseInt(limit);

    // 3. Configurar Filtros Dinámicos
    const whereClause = {};

    if (nombre) {
      // Buscar por nombre (LIKE %nombre%)
      whereClause.nombre = { [Op.like]: `%${nombre}%` };
    }

    if (minPrice) {
      whereClause.precio = { [Op.gte]: minPrice }; // Mayor o igual
    }
  }
}
```

```
// 4. Consulta a BD
const { count, rows } = await Product.findAndCountAll({
  where: whereClause,
  limit: limitNum,
  offset: offset,
  include: Category
});

// 5. Respuesta Estructurada
res.json({
  success: true,
  totalItems: count,
  totalPages: Math.ceil(count / limitNum),
  currentPage: parseInt(page),
  data: rows
});

} catch (error) {
  res.status(500).json({ error: error.message });
}
};
```